C PROGRAMMING MINI PROJECT REPORT

(4th Year Engineering Students)

Project Information

Field	Details
Course Name	C Programming Fundamentals
Project Title	Restaurant Billing System (CP-10)
Project Type	Console-Based Application
Programming Language	C
Development Environment	Code::Blocks / Dev C++ / Visual Studio Code
Version Control	Git & GitHub
Team Size	4 Students
Project Duration	2-3 Weeks
Difficulty Level	Beginner to Intermediate

1. Problem Statement

Section	Description
Existing Problem	Restaurants face manual inaccuracies and inefficiencies when taking customer orders, calculating sub-totals manually, computing exact tax allocations, and delivering formatted check summaries to visiting diners.
Proposed Solution	Develop a modular, functional console application containing menu arrays, standard dynamic item mapping fields, explicit input boundaries, and a dedicated automated calculation matrix computing 5% GST lines instantly alongside clean tabular command-line receipts.
Target Users	Restaurant Cashiers, Serving Waitstaff, and Restaurant Checkout Desk Operators
Expected Benefits	Elimination of manual computation math mistakes, rapid transactional speed, consistent tabular output formatting, and precise itemized order histories.

2. Project Objectives

Objective ID	Objective
OBJ-01	Implement a modular food catalog matrix housing fixed global item lists (Burger, Pizza, Sandwich, Cold Drink, French Fries) alongside predefined constant unit metrics.
OBJ-02	Design an automated, loop-driven user order collector that continuously stacks chosen quantities directly onto tracking structural records until checkout finalization.

Objective ID	Objective
OBJ-03	Establish strict programmatic navigation branches via C switch matrices offering targeted paths for Order Creation, Bill Generation, and Safe Closure.
OBJ-04	Build a precise calculation engine to map active index totals, factor exact 5% GST levels, and combine values into the ultimate grand total line.
OBJ-05	Construct an aligned tabular terminal output format to print item lists, clear column-based price extensions, and transactional signatures cleanly.

3. User Stories

User Story ID	User Story	Acceptance Criteria	Priority
US-01	As a Checkout Clerk, I want to review the full active dish item catalog directly inside the system terminal so I can view accurate base rates.	Displays the full item names with indexed identification digits and explicit base values matching configurations.	High
US-02	As a Cashier, I want to incrementally stack multiple quantities onto any selection without replacing previous sets so customer carts accumulate correctly.	Running variables receive simple compounding arithmetic accumulations upon every iteration loop safely.	High
US-03	As a Desk Operator, I want a dedicated option to end active cart gathering and immediately head back to the primary operational switch routing.	Selecting zero cleanly terminates loop prompts, carrying active state structures back down safely without context drops.	High
US-04	As a Customer, I want an aligned checkout printout displaying specific purchased names, explicit itemization counts, and exact tax calculations.	Produces clean block boundaries outlining subtotal aggregates, 5% tax values, and accurate payable lines.	High

4. Functional Requirements

Requirement ID	Requirement	Status
FR-01	Predefined Food Catalog Definition Module	
FR-02	Formatted Menu Console Presenter Function	
FR-03	Continuous Selection & Quantity Ingestion Loop	
FR-04	Out-of-Bounds Menu Choice Filtering Filter	
FR-05	Accumulative Dynamic Cart Tracking Array System	
FR-06	Aggregated Subtotal Math Processor	
FR-07	Fixed GST Macro Ratio Multiplier Engine	
FR-08	Tabular Column-Aligned Receipt Generator Blueprint	

Requirement ID	Requirement	Status
FR-09	Master Operational Switch-Driven Routing Shell	

5. Non-Functional Requirements

Category	Requirement
Performance	Arithmetic aggregates, sub-total equations, and complete receipt layout conversions must resolve within microseconds of function calls.
Reliability	State arrays must track zero changes continuously, ensuring clean cart clears on new operations.
Usability	Simple text interface commands with simple numeric entry workflows allow easy execution by non-technical operators.
Maintainability	Clean file architecture partitioning structural macro headers from system operations logic.
Portability	Uses standard ISO C parameters guaranteeing smooth processing on GCC, Clang, and MSVC compilers.

6. Project Modules

Module	Description	C Concepts Used
Operational Gateway Module	Acts as the terminal entry deck, launching master route selection frames and hosting the global state memory blocks.	Switch Matrices, Infinite Processing Loops, Data Structures
Catalog Display Module	Pulls fixed text arrays and mapping prices to produce standard vertical catalogs on screen.	Iterative Loops, Pointers, String Handling
Cart Entry Engine	Handles direct numerical user values, flags boundaries, and adds quantities directly into structural indexes.	Input Stream Formatting, Relational Flags, Memory Offsets
Financial Accounting Engine	Processes mathematical tracking arrays against fixed floating point costs to resolve accurate subtotals and 5% tax parameters.	Floating Math Equations, Accumulation Functions
Formatted Invoice Renderer	Converts active quantities into aligned multi-column summaries, adding footer margins and standard labels.	Output Formatting Modifiers, Structural Conditional Blocks

7. C Programming Concepts Used

Concept	Implementation
Global Macro Constant Directives	Utilized #define to enforce hardcoded limits for catalogs (#define SIZE 5) and fixed flat tax proportions (#define GST 0.05).
External Memory Linking Matrices	Implemented extern declarations inside header wrappers linking food data arrays and unit values cleanly across compilation steps.

Concept	Implementation
Multidimensional String Arrays	Constructed two-dimensional character maps (char food[SIZE][30]) to safely store dish strings like "French Fries" and "Burger".
Single Dimensional Float Arrays	Mapped corresponding transactional unit rates (float price[SIZE]) parallel to the item text catalog positions.
Tracking State Vectors	Initialized clean integer trackers (int quantity[SIZE]) across memory stacks to count accumulated ordering quantities.
Input Validation Controls	Enforced structural logical checks to verify if selection inputs fell outside safe programmatic parameters (choice < 1 || choice > SIZE).
Loop Control Mechanics	Integrated continuous processing sequences (while(1)) managed via custom break parameters to coordinate terminal loops.
Output Format Specifiers	Employed advanced printing flags (e.g., %-20s, %.2f) to produce perfectly aligned tabular alignment borders.

8. Expected Program Features

Feature	Description
Global Routing Gateway	Provides explicit commands for Order Creation, Bill Generation, and Program Exit paths.
Clean Order Resets	Ensures previous transaction states are cleared upon initializing a new order instance.
Live Catalog Rendering	Iterates directly through array matrices to present menu descriptions with clear price labels.
Boundary Input Interception	Catches out-of-bounds index inputs, prints specific errors, and resumes safely without code crashes.
Continuous Cart Accumulation	Allows continuous ordering inputs, appending quantities onto corresponding index values.
Tabular Receipt Generation	Filters non-zero entries to render clean invoices detailing quantities, item totals, 5% GST, and final totals.

9. Test Cases

Test Case ID	Scenario	Input	Expected Output	Status
TC-01	Master Menu Selection Validation	Option: 1	Launches takeOrder() context, presenting item menu listings.	Pass
TC-02	Out-of-Bounds Choice Filtering	Choice: 9	Displays "Invalid Choice." prompt and returns safely to input step.	Pass
TC-03	Item Entry Stacking Check	Choice: 2 (Pizza), Qty: 2	Registers "Item Added Successfully." and updates array tracker.	Pass

Test Case ID	Scenario	Input	Expected Output	Status
TC-04	Secondary Order Append Check	Choice: 4 (Cold Drink), Qty: 3	Appends values cleanly alongside prior selections inside memory.	Pass
TC-05	Order Completion Step	Choice: 0	Terminates selection routine loop and rolls back to core switch options.	Pass
TC-06	Financial Math & GST Verification	Option: 2	Computes Subtotal (500.00 + 150.00 = 650.00), GST (32.50), and Total (682.50).	Pass
TC-07	Zero Item Print Interception	Option: 2 (Empty Cart)	Outputs invoice layout boundaries containing 0.00 value balances.	Pass
TC-08	Graceful System Termination	Option: 3	Prints "Thank You." terminal notice and returns exit code 0.	Pass

10. Expected Deliverables

Deliverable	Description
Header Interface File	billing.h: Houses size parameters, tax macros, data declarations, and blueprints.
Operational Logic Source	billing.c: Contains menu views, item selectors, calculation engines, and invoice parsers.
Master Main Source	main.c: Provides entry execution, main loop orchestration, and navigation routing.
Project Overview Deck	PowerPoint slide deck outlining functional flows, logic architecture, and review profiles.
Analytical Test Book	Step-by-step diagnostic checklists mapping inputs directly against terminal expectations.
Verification Captures	Complete console screenshot logs confirming compilation and runtime checks.

Document Version: 1.2
Course: C Programming Fundamentals
Source Configuration: CP-10 Source Specifications (main.c, billing.c, billing.h)
Target Profile: 4th Year Engineering Student Evaluation Package